CST8116 Assignment 03 (24F)

Using Git within Eclipse.

Instructions

- It is expected that you have already reviewed and completed instructions for Git installation and configuration, as covered in the following Hybrids 05 and Hybrid 08 (which covers much of the same material as this lab).
    - **If you have not installed Git, or worked through the handouts for Hybrid 05 and 08, do so now.**

- Read this entire document before you begin. If you encounter problems and do not get the expected results, then you most certainly missed a step in the execution of the instructions. Undo what you've done, and start over, this time paying attention to the details. Implemented properly, this assignment should take you not more than 1½ hrs to complete (assuming you successfully completed Hybrids 05 and 08).

Tasks

Complete the tasks below completely and in the order given. Refer to the relevant hybrid activities resources (see above) if needed.

1. In your laptop, create a new folder named

   `C:\CST8116_Assignment_03_YourFullName`

   Using your first and last names for the full name (exactly as they are written in ACSIS/Brightspace—this applies wherever YourFullName appears in the instructions below). This will be used as the Git repository of Assignment 3.

2. In Eclipse, create a new Java project named Assignment 3

3. Create a new Git repository for the project, using the folder listed above in Step 1.

4. Verify that Git is configured to ignore the **bin** folder when adding files to the staging area.

5. Commit #1: Commit the project to the repository, with a description:

   *New Project Assignment03 by **YourFullName***

   ➔ Use your actual full name in the description
   ➔ Add a signed-off message, it should have your full name and college email.
   ➔ Note that at this point the project should not have any Java files.

6. Add a Java class named **Assignment03YourFullName**

   ➔ Add a main method into the class, have it print the text "Program by **YourFullName**"
   ➔ Use your actual full name in the class name and the printed text
   ➔ Verify that the project compiles and runs correctly
   ➔ Your Java file should be between 5 to 8 lines long, do not add comments yet.

7. Commit #2: Commit the Java file to the repository, with a description:

   Added source code file Assignment03YourFullName with method main

   ➔ Use your actual full name in the description
   ➔ Add a signed-off message as before
   ➔ Note that this commit should only include one Java file.

8. In your main() method, before the line that prints your name, insert the code below, exactly as written, including whatever errors are deliberately built into the code:

```
// The program rolls a 6-sided dice NUM_ROLLS times, records the results,
// and then counts how many times each number was rolled.

final int NUM_ROLLS = 10;

int[] rollResults = new int[NUM_ROLLS];

//countResults[i] represents the number of times i+1 is rolled
int[] countResults = new int[6];

// Roll the dice NUM_ROLLS times
for(int count = 0; count < NUM_ROLLS; ++count) {
    // Math.random() generates a random number between 0.0 and 1.0, multiply by 6 to get 0-5,
    // add 1 to get a number between 1 and 6 for each roll
    rollResults[count] = (int)(Math.random() * 6) + 1;

    // Increment the count for the rolled number
    countResults[rollResults[count] - 1]++;
}

// Display the results
for(int rollNum = 0; rollNum <= rollResults.length; ++rollNum) {  //Crashes
    System.out.printf("Result of roll #%d is: %d%n", rollNum, rollResults[rollNum]);
}

System.out.println("\nCount of each number rolled:");
for(int i = 0; i < countResults.length; ++i) {
    System.out.printf("Number %d was rolled %d times.%n", i + 1, countResults[i]);
}
```

   ➔ Note that at this point the program will run and terminate with an exception. **Do not fix this exception yet**.

9. Add programmer comments to the Java file, including a standard header at the top of the class

10. Commit #3: Commit the Java file to the repository, with an appropriate description of what changes you are committing.

    ➔ Add a signed-off message as before

11. Run the program; notice that it terminates with an exception.

12. Use the methods you have learned in this course to locate and correct the mistake(s) in the code. Ensure that the output is consistent with the logic. Correct the comments so they align with the execution of the working code.

13. Screenshot #1: Take a clear screenshot of the complete program's output, name it: **output.png**

14. Commit #4: Commit the Java file to the repository, with an appropriate description of what changes you are committing.

    ➔ The first description line must indicate what problem(s) you have corrected
    ➔ Add a signed-off message as before

15. Screenshot #2: Take a clear screenshot of the Git history of your project, name it: **history.png**

    ➔ The screenshot will include all the history columns: your commit: id, message, author, authored date, committer and committed date
    ➔ The screenshot will show the 4 commits you have done in the project. If you don't see all 4 commits, you may not be showing the history for the project, but for a specific folder/file.

16. Zip File of the Repository: Create a zip archive of the folder on C: drive you created under

    `C:\CST8116_Assignment_03_YourFullName`

Note that folders which do not contain a .git subfolder, containing complete information, will loose marks.

Submission Requirements

Use the rubric below to verify the files you are required to submit:

➔ **output.png**
➔ **history.png**
➔ **A zip file of your complete repository**

Once verified, submit all 3 files and look for a confirmation email on your submission.

Grading (10 points)

Repository	**Missing / Poor (0)**	**Below Expectations (3)**	**Meets Expectations (6)**
Git Repository	Missing from submission or only a project folder and not a full Git Repository.	Does not have requested folder name for repository, git log command may show inconsistencies when compared to screen shot. The bin folder content was committed and not ignored. The source file is not documented.	Has requested folder name for repository, git log command closely matches the history seen in the screen shot. The comments are corrected, consistent with the code.

Screenshots	**Missing / Poor (0)**	**Below Expectations (1)**	**Meets Expectations (2)**
output.png	Missing or poorly done	The program is not crashing but the output shows it wasn't fixed correctly: roll numbers may not make sense and/or 10 dice are not rolled.	Shows program executing successfully, student's full name as part of output.
history.png	Missing or poorly done	Shows some but not all of the requested commits, or commit text not correct.	Shows all four requested commits.